

P h i z u r a

On-the-Fly Music Recording with Method Proxies



Domenico Cipriani, Sebastian Jordan Montano, Stephane Ducasse

IWST 2026, Plovdiv, Bulgaria

inria



What is live coding?

- A increasingly popular performance practice: artists write and modify code in real time.
- The output is live : music, visuals, or poetry generated on the fly.
- An improvised art form: the code is the score, written in front of the audience
- Established field : the International Conference on Live Coding (ICLC) is in its 10th year
- Popular tools: Tidal Cycles, Strudel Sonic Pi, Orca, Mercury



Coypu: live music in Pharo

- A library and DSL for on-the-fly music programming, inside the Pharo environment
- First tool to bring live coding into the Smalltalk world
- Code is written and evaluated in a Playground
- It works as a live *score* for an audio *orchestra* - internal (Phausto DSP) or external via OSC or MIDI



The performance is what gets evaluated, when, and in what order

Inria



With Pharo we can capture your Performance in the Playground!



COOL! Because audio recording captures the sound, but throws away the process. And audio files are large and hard to share :{

Podemos Phizurear con los Method Proxies!



Inria



What Phizura does?

- Record the actual writing and evaluation of expressions
- Capture code executions AND keystrokes, with precise timestamps
- Replay the session as self-contained, executable Pharo code
- Faithful original timing : the script is alive, modifiable, *remixable*



The earlier approach (ESUG 2024)

- Built on Pharo's Announcer / Announcement classes -[the observer pattern] by Redwane Engels a student on internship for Evref team in Lille.
- The Coypu codebase was modified to fire an announcement on the dispatch of selected Sequencer and Performance methods.
- A recorder subscribed to those announcements and logged each with a timestamp.
- It worked - but with significant trade-offs...



We had to inject code on the method we wanted to record

```
to: aSymbol
```

```
| announcement |
```

```
announcement := SymbolAnnouncement new.
```

```
announcement symbol: aSymbol.
```

```
PerformanceRecorder uniqueInstance announce: announcement.
```

```
hexBeat
```

```
<onTheFlyEssentials>
```

```
| strings flat rhythm announcement |
```

```
thisContext sender selector asString = 'semibreves' ifFalse: [
```

```
announcement := RhythmAnnouncement new.
```

```
announcement rhythm: '', self asString, ' hexBeat'.
```

```
PerformanceRecorder uniqueInstance announce: announcement ].
```

Inria



Why the Announcer approach hurts

- Recording logic was scattered throughout the Coypu source
- Adding new performative methods on Coypu required injecting Announcements
- Tight coupling : recorder depended on Coypu's announcement classes
- Keystrokes, cursor, other tools : would need ever more invasive changes
- Any change to the evaluation pipeline risked silently breaking the hooks



What are Method Proxies

- A per-method instrumentation library for Pharo : 'message-passing control'
- Run user code before, after, or instead of a method - and during stack unwind
- Leaves the instrumented source code completely intact
- (Relatively) Simple API: subclass a handler (**MpHandler**), override a few hooks



Announcer vs Method Proxies

	Announcer	Method Proxies
Coypu changes	Announcer needed edits at every site	Method Proxies needs none
Coupling	Tight shared types	Fully external instrumentation
New events	Edit Coypu + define announcement	Intercept a method
Removal	Remove code from Coypu	Simply uninstall the proxies
Keystrokes	Not supported	Supported
Code emission	None before	2 modes (Performance and MoofLOD)

And now...

Phizura: architecture and implementation

Inria



Instrumentation

- PhizuraEvaluationInstrumentor picks the methods and installs an MpMethodProxy on each
- The two methods invoked during code evaluation are intercepted
- Covers all evaluation endpoints : every evaluation is captured
- Both proxies share one handler instance
- No Coypu class is modified, subclassed, or even mentioned



The handler

```
PhizuraHandler >> beforeExecutionWithReceiver: anObject arguments: anArrayOfArguments
| argument |
argument := anArrayOfArguments first.
recorder addRecord:
    (PhizuraEvaluationRecordEntry new
        time: DateAndTime now;
        expression: argument;
        cursorPosition: playgroundPage text cursorPosition;
        yourself)
```

The instrumentor



```
PhizuraEvaluationInstrumentor << initializeMethodProxies

evaluateAndGo := MpMethodProxy
    onMethod: SpCodeSelectionCommand >> #evaluate:andDo:
    handler: instrumentationHandler.

doItAndGo := MpMethodProxy
    onMethod: PhizuraPlaygroundPagePresenter >> #doEvaluateAndGoFor:
    handler: instrumentationHandler.
    proxyVariables addAll: { evaluateAndGo. doItAndGo }
```

The PhizuraPlaygroundPagePresenter add the keystrokes to the recorder



```
PhizuraPlaygroundPagePresenter >> recordKeyStrokes

self text eventHandler whenKeyUpDo: [ :event |

    PhiRecorderInstance addRecord:
    (PhizuraKeyEventRecordEntry

        time: DateAndTime now

        key: event key

        cursorPosition: self text
        cursorPosition) ]
```

The instrumentor



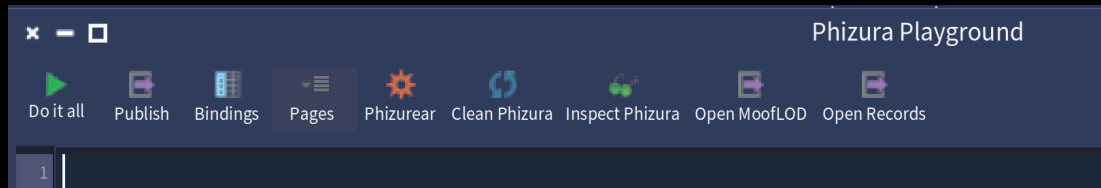
```
PhizuraEvaluationInstrumentor << initializeMethodProxies

evaluateAndGo := MpMethodProxy
    onMethod: SpCodeSelectionCommand >> #evaluate:andDo:
    handler: instrumentationHandler.

doItAndGo := MpMethodProxy
    onMethod: PhizuraPlaygroundPagePresenter >> #doEvaluateAndGoFor:
    handler: instrumentationHandler.
    proxyVariables addAll: { evaluateAndGo. doItAndGo }
```

The Phizura Playground

A custom subclass of `StPlaygroundPresenter` exposes ***doEvaluateAndGoFor:***, which receives the text directly.



We added five toolbar commands:

Phizurear (toggle recording);

Clean Phizura (reset);

Inspect Phizura (to inspect the record);

Open MoofLOD and ***Open Records*** - emit the two replay-script flavours



Code emission and replay

- Session ends as a time-ordered list of timestamped entries
- PhizuraCodeEmitter walks them, subtracts consecutive timestamps to recover delays
- Emits Pharo source separated by 'seconds wait' statements
- Wrapped in a forked block - waits don't freeze the UI, replay runs from the same Playground
- Double dispatch lets you add new formats or new entry types independently



Mode 1 - emitPerformanceCode

```
[  
  ep := EcoPhausto new.  
  1.2 seconds wait.  
  ep playFor: 128 bars.  
  1.5 seconds wait.  
  #(1 0 0 1 0 0 0 0) asSeq to: #kick.  
  1.8 seconds wait.  
  #rumba asRhythm to: #conga.  
] fork.
```

Keystrokes filtered out, only evaluations remain. Self-contained: any image with the same Coypu setup reproduces it verbatim.

Inria



Mode 2 - emitMoofLODCode

```
[  
  '1' toLocal: 'keyPressed'.  
  0.269059 seconds wait.  
  '6' toLocal: 'keyPressed'.  
  ...  
  16 plena to: #kick.  
  '16 plena to: #kick.' toLocal: 'evaluatedCommand'.  
] fork.
```

Same performance + plus OSC-compatible signals for every expression, keystroke and cursor move

Inria

Related work

- Tidal Cycles forum: stated the requirements, concluded with no tool existing
- Live Writing (Lee & Essl, 2015): records keystroke-level typing in a browser via editor hooks

Phizura instead uses reflective instrumentation - no editor or library changes

- SHARP: visualises the version history of code blocks, but no timing or replay

Complementary - SHARP keeps structure, Phizura keeps the temporal performance

Generality beyond music

- Instrumentation targets ALL Playground evaluations, not Coypu-specific code
- Visual live coding (Bloc, Roassal) could be recorded with no change to Phizura
- A general-purpose prototyping recorder - capture an exploration with order and pace
- Pedagogy: an instructor's session becomes a tiny text file students replay, pause, and edit



Limitations and future work

- Records evaluated expressions only - not un-evaluated edits, typing, or deletions
- Does not capture mouse actions (select, cut, paste via mouse)
- A single session at a time, held in a global variable
- Future: a public API for user-defined capture handlers on their own methods
- Future: recording across multiple Playgrounds for collaborative sessions



Conclusion

- Phizura is a non-intrusive recorder for live coding in Pharo.
- More modular, decoupled and extensible than the **Announcer** approach
- To our knowledge, this is the first tool to record live coding as it's typed.
- Method Proxies instrument the Playground's evaluation methods without touching Coypu
- Timestamped expressions are emitted as executable code that replays with faithful timing



**GRACIAS AMEEEEEG@S,
ME ENCANTA PHIZUREARRRRRR
PRRRRRRRRRR**



Enría